VISIÓN ARTIFICIAL · LIC. EN INTELIGENCIA ARTIFICIAL · 7° SEMESTRE

Práctica 1 · Procesamiento de imágenes

La imagen digital: matriz de píxeles, resolución, profundidad de bits, canales y tipos de imagen

Modalidad	Individual, en clase y junto con el docente (programación en vivo)
Duración	2 horas aprox.
Herramientas	Python 3.10 o superior · OpenCV · NumPy · Matplotlib · VS Code
Entregable	Los 3 scripts + reporte breve con capturas y respuestas (sección 8)

Objetivo

Que escribas, paso a paso, los scripts de Python que cargan una imagen (de un archivo o de la webcam), la muestran en pantalla y la transforman, y que con ellos entiendas **de qué está hecha una imagen digital**: una matriz de números con una resolución, una profundidad de bits y uno o varios canales.

Cómo está organizada la práctica

Cada paso es **un script nuevo que parte del anterior**: copias el archivo, conservas la parte común y le agregas algo más. Al terminar cada script lo ejecutas y comparas lo que ves con el **resultado esperado** de este documento.

Paso	Script	Qué agrega	Qué vas a ver
0	`prueba_entorno.py`	Comprobar la instalación	Versiones y dos ventanas de prueba
1	`p1_paso1_cargar_inspeccionar.py`	Cargar, mostrar, leer y cambiar píxeles	La imagen y su matriz de números
2	`p1_paso2_resolucion_bits.py`	Paso 1 + muestreo y cuantización	La imagen con menos píxeles y con menos niveles
3	`p1_paso3_tipos_canales.py`	Paso 1 + canales y tipos de imagen	Canales B, G, R; gris; binaria

En cada paso encontrarás tres apartados:

- **Lo que se verá:** el concepto, explicado en pocas líneas.
- **Qué tienes que hacer:** las instrucciones y el código clave.
- **Resultado esperado:** lo que debe aparecer en la consola y en pantalla.

No vamos a construir una interfaz gráfica: los resultados se muestran en **ventanas de OpenCV** (`cv2.imshow`) y en **figuras de Matplotlib** (varias imágenes con título en una sola ventana).

Contenido

- Objetivo
- Cómo está organizada la práctica
- 1. Preparar el entorno de trabajo
  - 1.1 Qué necesitas
  - 1.2 Carpeta de la práctica
  - 1.3 Crear y activar el entorno virtual
  - 1.4 Instalar las librerías
  - 1.5 Elegir el intérprete en VS Code
  - 1.6 Ejemplo mínimo: `prueba_entorno.py`
- 2. Conceptos clave
- 3. Paso 1 · Cargar, mostrar e inspeccionar la imagen
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 1
- 4. Paso 2 · Resolución espacial y profundidad de bits
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 2
- 5. Paso 3 · Tipos de imagen y canales de color
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 3
- 6. Retos (opcionales, suman puntos)
- 7. Errores frecuentes
- 8. Entregable
  - Referencia rápida

1. Preparar el entorno de trabajo

1.1 Qué necesitas

- **Python 3.10 o superior.** En Windows, descárgalo de python.org y en el instalador marca **"Add python.exe to PATH"**.
- **VS Code** con la extensión **Python** (de Microsoft).
- La carpeta `recursos/` que te comparte el docente, con las imágenes `astronauta.png` y `cafe.png`.
- Una webcam (opcional; la integrada de la laptop sirve).

Comprueba la versión en una terminal:

```
python --version        # macOS / Linux: python3 --version
```

1.2 Carpeta de la práctica

Crea una carpeta para la práctica (sin espacios ni acentos en el nombre) con esta estructura. Los scripts los irás creando tú:

```
practica01/
├── va/                                  <- entorno virtual (lo crea el paso 1.3)
├── recursos/
│   ├── astronauta.png
│   └── cafe.png
├── requirements.txt
├── prueba_entorno.py
├── p1_paso1_cargar_inspeccionar.py
├── p1_paso2_resolucion_bits.py
└── p1_paso3_tipos_canales.py
```

1.3 Crear y activar el entorno virtual

Un **entorno virtual** es una copia aislada de Python solo para este proyecto: las librerías que instales ahí no se mezclan con las de otros proyectos ni con las del sistema. Abre la carpeta `practica01` en VS Code y abre una terminal (*Terminal → New Terminal*):

Sistema	Crear (una sola vez)	Activar (cada vez que abras una terminal)
Windows (PowerShell)	`python -m venv va`	`va\Scripts\Activate.ps1`
Windows (cmd)	`python -m venv va`	`va\Scripts\activate.bat`
macOS / Linux	`python3 -m venv va`	`source va/bin/activate`

Sabrás que está activo porque la línea de la terminal empieza con `(va)`. Para salir del entorno: `deactivate`.

> **Si PowerShell dice que la ejecución de scripts está deshabilitada**, ejecuta una sola vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, responde **S** y vuelve a activar el entorno.

1.4 Instalar las librerías

Crea el archivo `requirements.txt` con este contenido:

```
opencv-python>=4.8
numpy>=1.24
matplotlib>=3.7
```

Con el entorno **activado**, instala:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Librería	Se importa como	Para qué la usamos
`opencv-python`	`import cv2`	Leer imágenes y video, convertir colores, redimensionar, umbralizar y mostrar ventanas
`numpy`	`import numpy as np`	Una imagen de OpenCV **es** un arreglo de NumPy: índices, *slicing*, operaciones
`matplotlib`	`import matplotlib.pyplot as plt`	Figuras con varias imágenes, títulos, histogramas y los números de los píxeles

> **Instala solo un paquete de OpenCV.** Si ya tenías `opencv-contrib-python` (incluye todo lo de `opencv-python` y más), no instales también `opencv-python`: los dos paquetes chocan entre sí. Si ya los tienes juntos, desinstala ambos (`pip uninstall opencv-python opencv-contrib-python`) e instala solo uno.

1.5 Elegir el intérprete en VS Code

Pulsa `Ctrl+Shift+P` → **Python: Select Interpreter** → elige el que dice `va` (`.\va\Scripts\python.exe`). Así VS Code reconoce `cv2` y el botón ▶ usa el entorno correcto. También puedes ejecutar todo desde la terminal con `python nombre_del_script.py`.

1.6 Ejemplo mínimo: `prueba_entorno.py`

Escribe este script, ejecútalo con `python prueba_entorno.py` y confirma que todo funciona. Construye una imagen **a mano** (una matriz de NumPy) y la muestra de dos formas:

```python
"""Prueba del entorno · Práctica 1

Comprueba que Python, NumPy, OpenCV y Matplotlib están instalados y que se pueden abrir ventanas.
Ejecutar (con el entorno virtual activado):  python prueba_entorno.py
"""
import sys                          # información del intérprete de Python

import cv2                          # OpenCV (paquete opencv-python)
import numpy as np                  # NumPy: arreglos numéricos
import matplotlib                   # Matplotlib: gráficas y figuras
import matplotlib.pyplot as plt     # interfaz de Matplotlib para dibujar figuras
```

```python

# 1) Versiones instaladas: si alguna importación falla, el script se detiene antes de llegar aquí
print("Python    :", sys.version.split()[0])
print("NumPy     :", np.__version__)
print("OpenCV    :", cv2.__version__)
print("Matplotlib:", matplotlib.__version__)

# 2) Una imagen hecha a mano: 100 filas x 300 columnas x 3 canales, todos los píxeles en 0 (negro)
imagen = np.zeros((100, 300, 3), dtype=np.uint8)
imagen[:, 0:100] = (255, 0, 0)    # primer tercio: azul  (OpenCV guarda los colores en orden B, G, R)
imagen[:, 100:200] = (0, 255, 0)  # segundo tercio: verde
imagen[:, 200:300] = (0, 0, 255)  # último tercio: rojo
print("Forma de la imagen:", imagen.shape)

# 3) Mostrarla con OpenCV: la ventana espera a que se pulse una tecla
cv2.imshow("Prueba OpenCV (pulsa una tecla)", imagen)
cv2.waitKey(0)
cv2.destroyAllWindows()

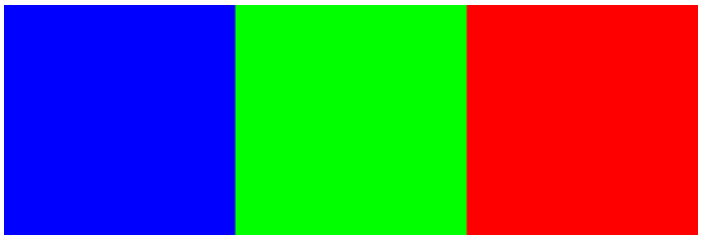
# 4) Mostrarla con Matplotlib: espera colores en orden R, G, B, así que se convierte primero
plt.imshow(cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB))
plt.title("Prueba Matplotlib (cierra la ventana)")
plt.show()

print("Entorno listo.")
```

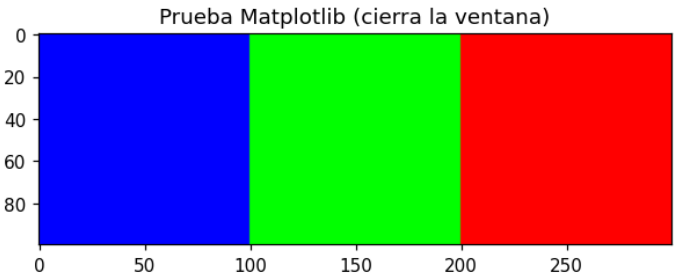
Resultado esperado. En la consola (tus versiones pueden ser más nuevas):

```
Python    : 3.13.13
NumPy     : 2.4.6
OpenCV    : 5.0.0
Matplotlib: 3.10.9
Forma de la imagen: (100, 300, 3)
Entorno listo.
```

Y dos ventanas, una tras otra: la de OpenCV (se cierra con cualquier tecla) y la de Matplotlib (se cierra con la ✕):



Ventana de OpenCV: azul, verde y rojo



Ventana de Matplotlib (con ejes)

> **Observa:** el primer tercio se ve **azul** aunque escribimos `(255, 0, 0)`. OpenCV guarda los colores en orden **B, G, R** (no R, G, B). Matplotlib sí espera R, G, B, por eso convertimos con `cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB)` antes de `plt.imshow`.

2. Conceptos clave

- **Imagen digital:** una función f(x, y) **muestreada** (dividida en una rejilla de píxeles) y **cuantizada** (cada píxel guarda un número entero de un rango limitado).
- **Píxel:** elemento mínimo de la imagen. En gris guarda **1 número** (intensidad); a color guarda **3** (B, G, R).
- `shape`**:** `(alto, ancho)` en gris y `(alto, ancho, canales)` a color. **Primero van las filas (alto), luego las columnas (ancho).**
- **Acceso a un píxel:** `img[y, x]` = `img[fila, columna]`. El origen (0, 0) está **arriba a la izquierda**; `y` crece hacia abajo.
- `uint8`**:** entero sin signo de 8 bits, de **0 a 255**. 0 = negro, 255 = máxima intensidad. Si una suma pasa de 255, **se desborda** (da la vuelta: 250 + 10 = 4).
- **Resolución espacial:** cuántos píxeles tiene la imagen (ancho × alto). Más píxeles = más detalle.
- **Profundidad de bits:** cuántos bits tiene cada píxel por canal. *b* bits dan 2^*b* niveles: 8 bits = 256, 1 bit = 2.
- **Tipos de imagen:** a **color** (3 canales), en **gris** (1 canal, 256 niveles) y **binaria** (1 canal, solo 2 valores).



3. Paso 1 · Cargar, mostrar e inspeccionar la imagen

Lo que se verá

Cómo obtener una imagen (de un archivo o de la webcam), qué propiedades tiene (`shape` , `dtype` , rango, memoria), cómo mostrarla con `cv2.imshow` , cómo leer y modificar píxeles, y cómo **verla como matriz**: cada píxel de una imagen a color tiene 3 valores (B, G, R) y cada píxel en gris tiene uno.

Qué tienes que hacer

Crea `p1_paso1_cargar_inspeccionar.py` .

1. Sección `[BASE]` . Son las importaciones y funciones que **reutilizarás en los pasos 2 y 3**. Escríbela completa:

```
import sys                          # sys.argv (argumentos) y sys.exit (terminar con un mensaje)
from pathlib import Path            # rutas que funcionan igual en Windows, macOS y Linux

import cv2                          # OpenCV: leer, mostrar y procesar imágenes
import numpy as np                  # NumPy: una imagen de OpenCV ES un arreglo de NumPy
import matplotlib.pyplot as plt     # Matplotlib: figuras con varios paneles, títulos y números

# ============================================================================
# [BASE] Configuración y funciones que se reutilizan en los pasos 2 y 3
# ============================================================================
CARPETA = Path(__file__).resolve().parent                # carpeta donde está este script
RUTA_IMAGEN = CARPETA / "recursos" / "astronauta.png"    # imagen de trabajo (puedes cambiarla por
una tuya)


def cargar_imagen(ruta):
    """Lee una imagen del disco. OpenCV la devuelve en orden de canales B, G, R y tipo uint8."""
    imagen = cv2.imread(str(ruta))                       # str(): imread recibe texto, no un objeto
Path
    if imagen is None:                                   # imread NO lanza error: devuelve None si
falla
        sys.exit(f"No se pudo leer la imagen: {ruta}")   # terminamos con un mensaje claro
    return imagen


def capturar_webcam(indice=0):
    """Abre la cámara y devuelve una foto. ESPACIO = capturar · ESC = cancelar."""
    camara = cv2.VideoCapture(indice)                    # 0 = primera cámara del equipo
    if not camara.isOpened():                            # ¿la cámara existe y está libre?
        sys.exit("No se pudo abrir la cámara (¿la usa otra aplicación?)")
    foto = None                                          # aquí guardaremos la foto capturada
    while True:                                          # un video es una secuencia de imágenes
(frames)
        ok, frame = camara.read()                        # ok = True si llegó un frame; frame =
imagen BGR
        if not ok:                                       # la cámara dejó de entregar imágenes
            break
        cv2.imshow("Webcam - ESPACIO captura, ESC cancela", frame)   # mostramos el frame actual
        tecla = cv2.waitKey(1) & 0xFF                    # espera 1 ms por una tecla (y refresca la
ventana)
        if tecla == 32:                                  # 32 = código ASCII de la barra espaciadora
            foto = frame.copy()                          # copia: el siguiente read() reutiliza la
memoria
            break
        if tecla == 27:                                  # 27 = código de la tecla ESC
```

```
            break
    camara.release()                                    # liberar la cámara para otras aplicaciones
    cv2.destroyAllWindows()                             # cerrar la ventana del video
    if foto is None:
        sys.exit("No se capturó ninguna foto")
    return foto


def obtener_imagen():
    """Elige la fuente: webcam si se ejecutó con --camara; si no, el archivo RUTA_IMAGEN."""
    if "--camara" in sys.argv:                          # sys.argv = lista de palabras escritas al ejecutar
        return capturar_webcam()
    return cargar_imagen(RUTA_IMAGEN)


def imprimir_info(nombre, img):
    """Imprime las propiedades básicas de una imagen (sirve para color, gris o binaria)."""
    alto, ancho = img.shape[:2]                         # shape = (filas, columnas[, canales])
    canales = 1 if img.ndim == 2 else img.shape[2]      # ndim = 2 -> matriz (gris); 3 -> cubo (color)
    print(f"{nombre:<10} shape={str(img.shape):<15} ancho={ancho:<4} alto={alto:<4} "
          f"canales={canales}  dtype={img.dtype}  min={img.min():>3}  max={img.max():>3}  "
          f"memoria={img.nbytes / 1024:.0f} KB")


def a_rgb(img):
    """Matplotlib espera R, G, B; OpenCV entrega B, G, R. Las imágenes en gris se dejan igual."""
    return img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


# ============================================================================
```

Puntos importantes de la sección base:

- `cv2.imread` **no lanza error** si la ruta está mal: devuelve `None`. Por eso lo comprobamos.
- La ruta se construye a partir de la carpeta del script (`Path(__file__)`), así funciona sin importar desde dónde ejecutes.
- `capturar_webcam()` lee frames en un ciclo; `cv2.waitKey(1)` refresca la ventana y devuelve la tecla pulsada (32 = espacio, 27 = Esc).

2. Programa principal. Debajo de la sección base, agrega un bloque `if __name__ == "__main__":` que haga lo siguiente:

a. Obtener la imagen, convertirla a gris e imprimir sus propiedades:

```
imagen = obtener_imagen()
imprimir_info("color", imagen)
gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
imprimir_info("gris", gris)
```

b. Leer el píxel de la fila 100, columna 200, a color y en gris:

```
y, x = 100, 200
b, g, r = imagen[y, x]
print(f"color -> B={b}  G={g}  R={r}")
print(f"gris  -> {gris[y, x]}")
```

c. Pintar un cuadro rojo de 40×40 píxeles **sobre una copia** (rojo en OpenCV es `(0, 0, 255)`):

```
marcada = imagen.copy()
marcada[y:y + 40, x:x + 40] = (0, 0, 255)
```

d. Mostrar las tres imágenes con `cv2.imshow("titulo", img)`, esperar una tecla con `cv2.waitKey(0)` y cerrar con `cv2.destroyAllWindows()`.

e. Recortar una región de 6×6 píxeles que empiece en `(y0, x0) = (382, 131)` e imprimirla en gris y por canal:

```
N = 6
y0, x0 = 382, 131
region_color = imagen[y0:y0 + N, x0:x0 + N]
region_gris = gris[y0:y0 + N, x0:x0 + N]
print(region_gris)
print(region_color[:, :, 0])      # canal B; usa 1 para G y 2 para R
```

f. Hacer una figura de Matplotlib con 3 paneles: la imagen completa con la región marcada, la región a color con los 3 números de cada píxel y la región en gris con su número. Para escribir el valor dentro de cada píxel se recorre la región con `np.ndenumerate`:

```
fig, ejes = plt.subplots(1, 3, figsize=(15, 5.5), layout="constrained")
ejes[2].imshow(region_gris, cmap="gray", vmin=0, vmax=255)
for (fila, col), valor in np.ndenumerate(region_gris):
    ejes[2].text(col, fila, str(valor), ha="center", va="center")
plt.show()
```

Ejecuta: `python p1_paso1_cargar_inspeccionar.py`. Para usar la webcam: `python p1_paso1_cargar_inspeccionar.py --camara` (pulsa **espacio** para tomar la foto).

Resultado esperado

En la consola:

```
== 1. Propiedades ==
color      shape=(512, 512, 3)   ancho=512  alto=512  canales=3  dtype=uint8  min=  0  max=255  memoria=768 KB
gris       shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB

== 3. Píxel en (y=100, x=200) ==
color -> B=17  G=57  R=81
gris  -> 60

Pulsa cualquier tecla sobre una ventana de OpenCV para continuar...

== 6. Región 6x6 a partir de (y=382, x=131) ==
Gris (1 número por píxel):
[[121 117 112  85  89  99]
 [117 121 113  82  95  97]
 [118 114 110  76  92  96]
 [125 119 109  71  95  95]
 [125 123 103  68  97  85]
 [125 118 103  67 101  74]]
Canal B (region_color[:, :, 0]):
[[ 56  64 118 116 145 176]
 [ 51  69 130 115 153 174]
 [ 53  61 129 110 154 176]
 [ 64  67 133 101 157 173]
 [ 66  79 129 101 159 162]
 [ 62  70 126 104 168 148]]
Canal G (region_color[:, :, 1]):
```

```
 [[87 81 70 43 60 76]
  [83 87 71 40 67 74]
  [84 79 67 35 65 71]
  [91 85 65 30 67 72]
  [90 88 59 26 71 61]
  [91 82 59 25 72 48]]
Canal R (region_color[:, :, 2]):
[[213 207 191 154 126 116]
  [208 206 190 151 128 113]
  [209 204 187 145 122 115]
  [214 206 185 139 125 112]
  [215 209 180 138 126 104]
  [215 206 179 137 133  96]]
```

Tres ventanas de OpenCV (se cierran con cualquier tecla):



Original (color, 3 canales)

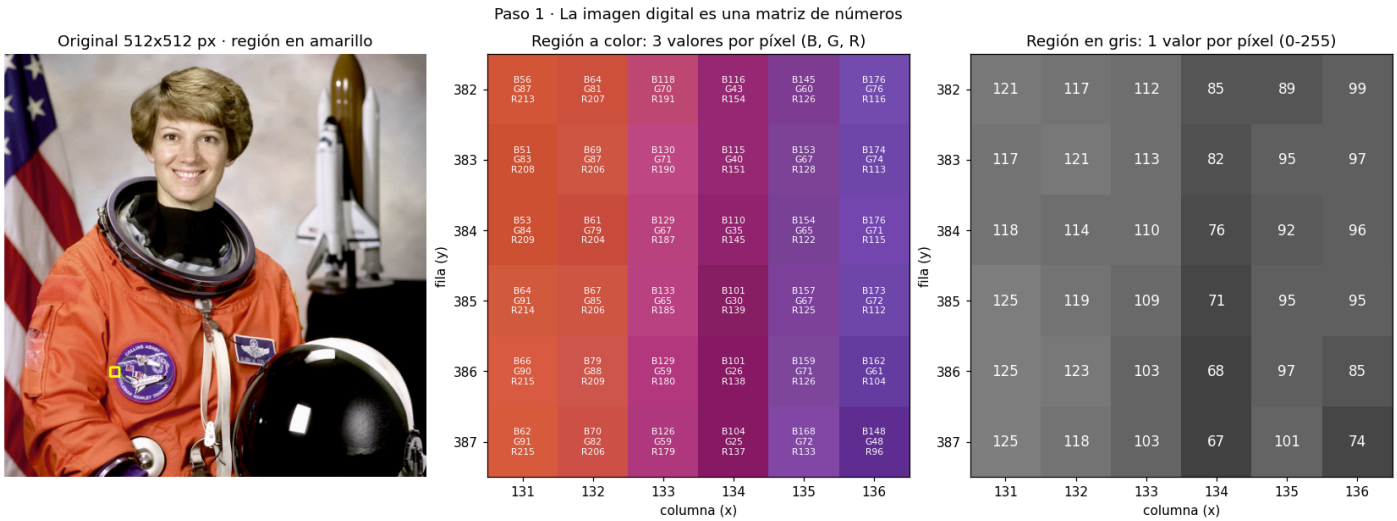


Escala de grises (1 canal)



Copia con un cuadro rojo en (y=100, x=200)

Y después la figura de Matplotlib con la matriz de píxeles:



La región 6×6 a color (B, G, R por píxel) y en gris (un valor por píxel)

Preguntas del paso 1

1. La imagen tiene `shape = (512, 512, 3)`. ¿Qué significa cada número? ¿Cuántos números guarda en total?

2. ¿Por qué la imagen a color ocupa exactamente el triple de memoria que la imagen en gris?
3. El cuadro rojo empieza en `y=100, x=200`. ¿Quedó a 100 píxeles del borde **superior** o del **izquierdo**? ¿Por qué?
4. ¿Qué pasaría con la imagen original si escribieras `marcada = imagen` (sin `.copy()`)? Pruébalo.
5. En la región 6×6, el píxel naranja de la izquierda tiene R alto y el morado de la derecha B alto. ¿Cuál de los dos es más claro en gris? Explícalo con la fórmula del gris.

4. Paso 2 · Resolución espacial y profundidad de bits

Lo que se verá

Una imagen digital se obtiene en dos pasos, y cada uno pierde información:

- **Muestreo → resolución espacial.** La escena se divide en una rejilla de píxeles. Con **menos píxeles** se pierde el detalle fino: los bordes se vuelven escalones y los objetos pequeños desaparecen.
- **Cuantización → profundidad de bits.** Cada píxel guarda un número de un rango limitado. Con **menos niveles**, las zonas de cambio suave se convierten en franjas planas con bordes que no existen en la escena (**falsos contornos**).

Bits por píxel	Niveles (2^b)	Memoria de una imagen gris de 512×512
8	256	256 KB
4	16	128 KB
1	2	32 KB

Qué tienes que hacer

1. **Copia** `p1_paso1_cargar_inspeccionar.py` como `p1_paso2_resolucion_bits.py`.
2. **Conserva** la sección `[BASE]` y **borra** todo el bloque `if __name__ == "__main__":` del paso 1.
3. **Agrega** estas dos funciones debajo de la sección base:

```python
def reducir_resolucion(img, factor):
    """Muestreo: deja 1 pixel por cada bloque de factor x factor.
    Devuelve la imagen pequeña y la misma ampliada al tamaño original para compararlas lado a lado."""
    alto, ancho = img.shape[:2]
    pequena = cv2.resize(img, (ancho // factor, alto // factor),     # OJO: dsize va como (ANCHO, ALTO)
                         interpolation=cv2.INTER_AREA)               # INTER_AREA promedia el bloque (reducir)
    ampliada = cv2.resize(pequena, (ancho, alto),                    # regresamos al tamaño original...
                          interpolation=cv2.INTER_NEAREST)           # ...repitiendo cada píxel: se ven bloques
    return pequena, ampliada


def reducir_bits(img, bits):
    """Cuantización: deja solo 2**bits niveles por canal y los reparte entre 0 y 255 para poder verlos."""
    niveles = 2 ** bits                                      # 8 bits -> 256 niveles; 1 bit -> 2 niveles
    paso = 256 // niveles                                    # cuántos valores originales caen en cada nivel
    indice = img // paso                                     # división entera: nivel de cada píxel (0..niveles-1)
    return (indice * (255 // (niveles - 1))).astype(np.uint8)   # nivel -> intensidad visible (0..255)
```

4. **Escribe el nuevo programa principal:**

a. Obtén la imagen y su versión en gris, como en el paso 1.
b. **Resolución:** para cada factor de `[1, 2, 4, 8, 16, 32]`, llama a `reducir_resolucion(imagen, factor)`, imprime la resolución, los píxeles y la memoria (`pequena.nbytes`), y muestra la versión `ampliada` en un panel de una figura de 2×3.
c. **Bits:** para cada valor de `[8, 4, 3, 2, 1]`, llama a `reducir_bits` con la imagen gris **y** con la imagen a color. Imprime cuántos valores distintos quedaron (`len(np.unique(g))`) y la memoria teórica (`píxeles × bits / 8`). Muestra una figura de 3 filas: gris, su histograma y color.
d. Termina con un solo `plt.show()` para ver las dos figuras.

Esta plantilla sirve para mostrar **varias imágenes en una figura** (la usarás también en el paso 3):

```
fig, ejes = plt.subplots(2, 3, figsize=(13, 9), layout="constrained")   # 2 filas x 3 columnas
fig.suptitle("Título de la figura")
for eje, factor in zip(ejes.flat, [1, 2, 4, 8, 16, 32]):                # un panel por factor
    pequena, ampliada = reducir_resolucion(imagen, factor)
    eje.imshow(a_rgb(ampliada))                                          # a color: convertir a RGB
    eje.set_title(f"{pequena.shape[1]}x{pequena.shape[0]} px")
    eje.axis("off")
# Para el histograma de una imagen gris g:  eje.hist(g.ravel(), bins=256, range=(0, 256))
plt.show()
```

> **Cuidado con `cv2.resize`:** el tamaño se da como `(ancho, alto)`, al revés que `shape`, que es `(alto, ancho)`.

Resultado esperado

En la consola:

```
== Imagen de trabajo ==
color      shape=(512, 512, 3)   ancho=512  alto=512  canales=3  dtype=uint8  min=  0  max=255  memoria=768 KB
gris       shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB

== 1. Resolución espacial (muestreo), imagen a color de 8 bits por canal ==
factor  resolución   píxeles    memoria
     1     512x512    262144   768.0 KB
     2     256x256     65536   192.0 KB
     4     128x128     16384    48.0 KB
     8       64x64      4096    12.0 KB
    16       32x32      1024     3.0 KB
    32       16x16       256     0.8 KB

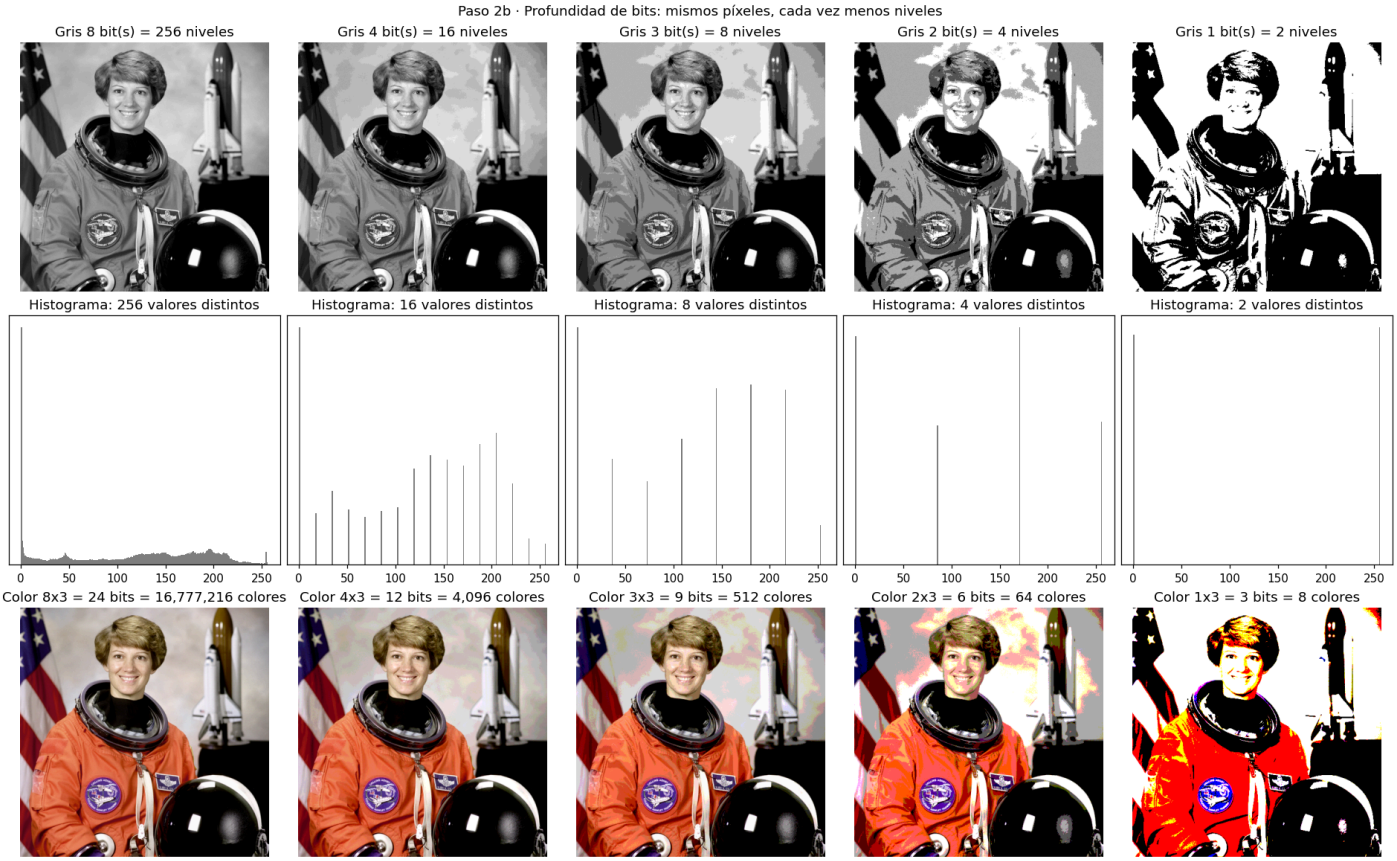
== 2. Profundidad de bits (cuantización), imagen en gris ==
bits  niveles  valores distintos  memoria teórica  memoria real (uint8)
   8      256               256          256 KB               256 KB
   4       16                16          128 KB               256 KB
   3        8                 8           96 KB               256 KB
   2        4                 4           64 KB               256 KB
   1        2                 2           32 KB               256 KB
```

Figura 1, resolución espacial. Los 8 bits por canal se mantienen y solo cambia el número de píxeles:



Resolución espacial: de 512×512 a 16×16 píxeles

Figura 2, profundidad de bits. Los 512×512 píxeles se mantienen y solo cambia el número de niveles. Observa los **falsos contornos** en el fondo con 3 y 2 bits, y cómo el histograma se reduce a unas cuantas barras:



Profundidad de bits: de 256 a 2 niveles en gris y a color

Preguntas del paso 2

1. ¿A partir de qué resolución ya no reconoces el rostro? ¿Y el parche de la misión?
2. ¿Cuántas veces menos memoria ocupa la imagen cada vez que divides el ancho y el alto entre 2? ¿Por qué no es “2 veces”?
3. ¿Con cuántos bits empiezas a notar los falsos contornos? ¿En qué zonas de la imagen aparecen primero y por qué ahí?
4. La memoria real (`nbytes`) **no baja** al reducir los bits. ¿Por qué? ¿Qué habría que hacer para que sí bajara?
5. ¿Qué afecta más a la información de la imagen: pasar de 512×512 a 128×128 (16 veces menos píxeles) o de 8 a 2 bits (4 veces menos bits)? Justifica con las figuras.

5. Paso 3 · Tipos de imagen y canales de color

Lo que se verá

Tipo	Canales	`shape`	Valores por píxel	Uso típico
Color	3 (B, G, R)	`(alto, ancho, 3)`	3 números de 0 a 255	Fotografía, segmentar por color
Gris	1	`(alto, ancho)`	1 número de 0 a 255	Casi todo el procesamiento: bordes, filtros, umbrales
Binaria	1	`(alto, ancho)`	Solo 0 o 255	Máscaras, segmentación, contar y medir objetos

- **Canal:** cada canal de una imagen a color es una imagen en gris que dice **cuánto** hay de ese color en cada píxel. Donde el canal R es claro, hay mucho rojo.
- **Gris:** no es el promedio de los canales, sino una **suma ponderada**: `gris = 0.299·R + 0.587·G + 0.114·B`. El ojo humano es más sensible al verde.
- **Binaria:** se obtiene con un **umbral** *T*: si `gris > T` el píxel vale 255 (blanco); si no, 0 (negro). Sirve como **máscara** para quedarse solo con una parte de la imagen.

Qué tienes que hacer

1. **Copia** el paso 1 como `p1_paso3_tipos_canales.py`, conserva la sección `[BASE]` y borra el programa principal anterior.
2. **Color = 3 canales.** Separa los canales e imprime la información de cada uno:

```
b, g, r = cv2.split(imagen)                # 3 matrices de alto x ancho
imprimir_info("canal B", b)                # repite para g y r
ceros = np.zeros_like(b)
solo_r = cv2.merge([ceros, ceros, r])      # solo el canal rojo, en su color
reconstruida = cv2.merge([b, g, r])        # comprueba con np.array_equal que es igual a la original
```

Haz una figura de 2×4. Arriba: original y cada canal en gris (`cmap="gray"`). Abajo: `plt.imshow(imagen)` **sin convertir** (para ver el error de BGR) y cada canal en su color.

3. **Gris = 1 canal.** Calcula el gris de cuatro maneras y muéstralas en una figura de 1×4:

```
gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
gris_formula = np.round(0.114 * b + 0.587 * g + 0.299 * r).astype(np.uint8)
promedio = ((b.astype(np.uint16) + g + r) // 3).astype(np.uint8)
promedio_mal = (b + g + r) // 3            # ¡a propósito! uint8 se desborda
```

Imprime la diferencia máxima entre `gris` y `gris_formula` con `cv2.absdiff(...).max()`, y para el píxel `(60, 250)` compara la suma real de B + G + R con la suma en `uint8`.

4. **Binaria = 2 valores.** Umbraliza el gris y úsalo como máscara:

```python
_, binaria = cv2.threshold(gris, 127, 255, cv2.THRESH_BINARY)
print(np.unique(binaria))                  # debe dar [  0 255]
blancos = cv2.countNonZero(binaria)
recorte = cv2.bitwise_and(imagen, imagen, mask=binaria)
```

Muestra en una figura de 1×5: el gris, la binaria con T = 64, 127 y 192, y el `recorte`.

5. **Resumen.** Imprime una tabla con `shape`, `dtype`, valores distintos y memoria de las imágenes a color, en gris y binaria, y muéstralas lado a lado en una figura de 1×3. Para contar los colores distintos: `len(np.unique(imagen.reshape(-1, 3), axis=0))`.

Resultado esperado

En la consola (el aviso `RuntimeWarning: overflow` al sumar el píxel es **esperado**: es justo el desbordamiento que queremos ver):

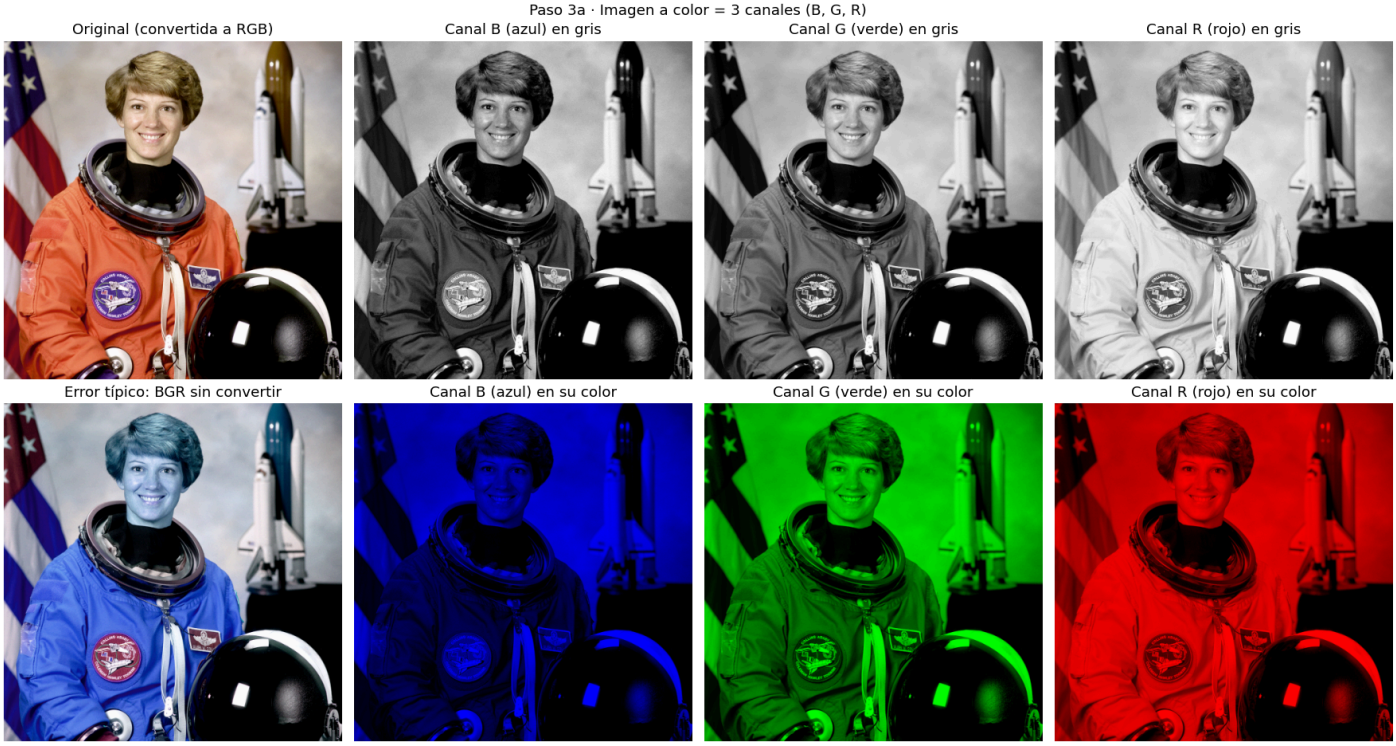
```
== Imagen de trabajo ==
color      shape=(512, 512, 3)   ancho=512  alto=512  canales=3  dtype=uint8  min=  0  max=255  memoria=768 KB

== 1. Canales de la imagen a color ==
canal B    shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB
canal G    shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB
canal R    shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB
merge([b, g, r]) es idéntica a la original: True

== 2. Escala de grises ==
gris       shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB
diferencia máxima cvtColor vs fórmula: 1
píxel (y=60, x=250): B=152 G=192 R=203
  suma real = 547   suma en uint8 = 35  <- desbordamiento (módulo 256)

== 3. Imagen binaria ==
binaria    shape=(512, 512)      ancho=512  alto=512  canales=1  dtype=uint8  min=  0  max=255  memoria=256 KB
valores distintos: [  0 255]
píxeles blancos: 133199 de 262144 (50.8 %)
máscara NumPy: bool (512, 512) -> coincide: True

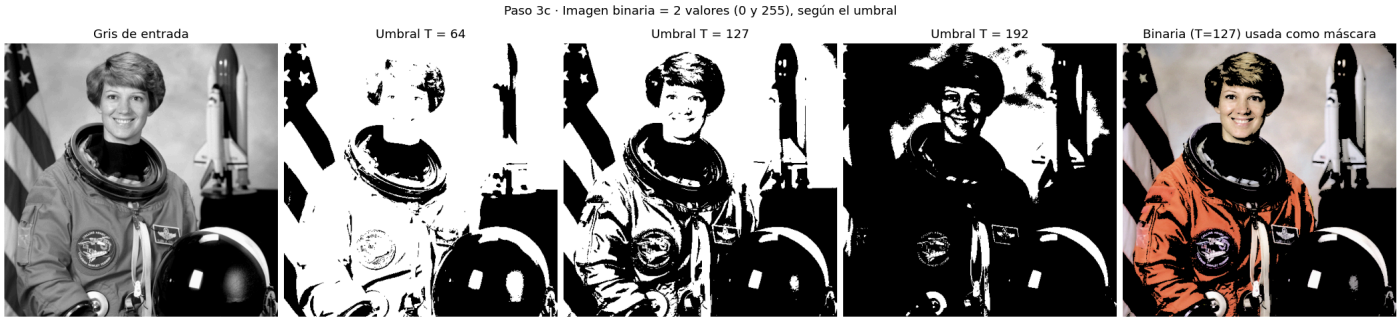
== 4. Resumen de tipos de imagen ==
tipo     shape           dtype  valores distintos   memoria
color    (512, 512, 3)   uint8             113382    768 KB
gris     (512, 512)      uint8                256    256 KB
binaria  (512, 512)      uint8                  2    256 KB
```



Paso 3a: la imagen a color y sus tres canales



Paso 3b: cuatro formas de calcular el gris; la última muestra el desbordamiento de uint8



Paso 3c: la binaria depende del umbral y sirve como máscara



Paso 3d: resumen de los tres tipos de imagen

Preguntas del paso 3

1. El traje es naranja. ¿En qué canal se ve **más claro** y en cuál **más oscuro**? ¿Por qué?
2. ¿Por qué al mostrar la imagen BGR sin convertir el traje se ve azul y la piel azulada?
3. ¿Por qué la imagen `promedio_mal` se ve con manchas oscuras justo en las zonas **claras**?
4. ¿Qué umbral separa mejor a la astronauta del fondo? ¿Existe un umbral que lo haga perfecto? ¿Por qué?
5. La binaria ocupa la misma memoria que el gris, aunque solo tiene 2 valores. ¿Cuántos bits por píxel bastarían?

6. Retos (opcionales, suman puntos)

1. **Tu propia imagen.** Ejecuta los tres pasos con `--camara` o cambiando `RUTA_IMAGEN` por una foto tuya. Ajusta `y0, x0` del paso 1 para que la región caiga en un borde interesante.
2. **Umbral interactivo.** En el paso 3 crea una ventana con `cv2.namedWindow` y una barra con `cv2.createTrackbar("T", "Binaria", 127, 255, lambda v: None)`. En un ciclo, lee el valor con `cv2.getTrackbarPos`, umbraliza y muestra la binaria hasta que se pulse Esc.
3. **Interpolación.** En `reducir_resolucion`, cambia `INTER_NEAREST` por `INTER_LINEAR` y `INTER_CUBIC` al ampliar. ¿Qué cambia? ¿Se recupera el detalle perdido?
4. **Video en vivo.** Aplica `reducir_bits` a cada frame de la webcam (con el ciclo de `capturar_webcam`) y muestra el original y el cuantizado en dos ventanas.

7. Errores frecuentes

Síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'cv2'`	El entorno no está activado, o VS Code usa otro intérprete	Activa `va` y elige su intérprete (sección 1.5)
`AttributeError: 'NoneType' object has no attribute 'shape'`	`cv2.imread` no encontró el archivo	Revisa la ruta y el nombre; usa `Path(__file__)`
La ventana de OpenCV no responde o se cierra sola	Falta `cv2.waitKey`	Pon `cv2.waitKey(0)` después de `imshow`
Colores raros en Matplotlib (rojo ↔ azul)	Imagen BGR mostrada como RGB	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
Imagen gris se ve con colores (verde/morado)	`plt.imshow` usa un mapa de color por defecto	`cmap="gray", vmin=0, vmax=255`
`cv2.resize` deja la imagen girada o deformada	Se pasó `(alto, ancho)`	El tamaño va como `(ancho, alto)`
La webcam no abre	Otra app la usa (Zoom, Teams, Cámara)	Ciérrala; prueba `cv2.VideoCapture(1)`

8. Entregable

Sube a la plataforma del curso **un** `.zip` con:

1. Los tres scripts `p1_paso1_...py`, `p1_paso2_...py` y `p1_paso3_...py`, funcionando.
2. Un **reporte PDF** (máximo 4 páginas) con:
   - Las capturas de las figuras de cada paso, hechas **con una imagen tuya** (webcam o foto propia), no con la astronauta.
   - Las respuestas a las preguntas de los pasos 1, 2 y 3.
   - Si hiciste retos, una captura de cada uno.

Criterio	Puntos
Los 3 scripts corren sin errores y muestran los resultados esperados	4
Código ordenado y comentado; cada paso parte del anterior	2
Respuestas correctas y justificadas con las figuras	3
Reporte claro, con capturas de una imagen propia	1
Retos (extra)	hasta +1

Referencia rápida

Función	Qué hace
`cv2.imread(ruta)`	Lee una imagen (BGR, `uint8`); devuelve `None` si falla
`cv2.VideoCapture(0)` / `.read()` / `.release()`	Abre la cámara / lee un frame / la libera
`cv2.imshow(titulo, img)` + `cv2.waitKey(0)`	Muestra una ventana y espera una tecla
`cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`	Convierte de color a gris (también `COLOR_BGR2RGB`)
`cv2.resize(img, (ancho, alto), interpolation=...)`	Cambia la resolución
`cv2.split(img)` / `cv2.merge([b, g, r])`	Separa / une canales
`cv2.threshold(gris, T, 255, cv2.THRESH_BINARY)`	Umbraliza (devuelve `(T, binaria)`)
`cv2.bitwise_and(img, img, mask=m)`	Conserva solo donde la máscara es distinta de 0
`img.shape`, `img.dtype`, `img.nbytes`	Forma, tipo de dato y bytes
`img[y, x]`, `img[y1:y2, x1:x2]`, `img[:, :, c]`	Un píxel, una región, un canal
`np.unique(img)`, `np.zeros_like(img)`	Valores distintos, matriz de ceros del mismo tamaño
`plt.subplots(f, c, layout="constrained")` + `plt.show()`	Figura con f×c paneles